Python Beginner Guide

2026 Class Setup

Jaewoong Lee

Ulsan National Institute of Science and Technology

jwlee230@unist.ac.kr

2026 Class

Introduction

This guide assumes the 2026 class environment:

1. A terminal on a course computer, local laptop, or remote server
2. Python 3.14.5 selected for this repository
3. A text editor for writing `.py` files
4. Basic shell commands such as `cd`, `ls`, and `mkdir`

The repository includes `.python-version` so pyenv-compatible setups can select the same Python version automatically.

Class Python Setup

Check the interpreter before running examples:

```
python3 --version
```

For this class, the expected version is:

```
Python 3.14.5
```

The examples use `python3`. If your pyenv setup also makes `python` point to Python 3.14.5, that command is fine too.

Overview

1. Python?
2. Basic I/O
3. Data Types
4. File I/O
5. if, for, and while
6. Functions
7. Handling Exceptions
8. Modules

Python?

Figure: Logo of Python

Python is an interpreted, high-level programming language created by Guido van Rossum. It is widely used for automation, data analysis, web services, scientific computing, and teaching programming.

Interpreted?

A compiled language is usually translated into an executable file before you run it.

An interpreted language is run by another program called an *interpreter*. In class, the interpreter is the `python3` program.

Python still creates bytecode internally, but beginners normally run code by giving a `.py` file to the interpreter.

How to Run Python?

There are two common ways to run Python:

```
$ python3
Python 3.14.5
Interactive prompt
>>> print("Hello world")
Hello world
>>>
```

(a) Python interpreter

```
$ python3 hello.py
Hello world
```

(b) Python script

We will prefer scripts because a file can be saved, reviewed, rerun, and shared.

First Script

Create a file named `hello.py`:

```
print("Hello world")
```

Run it from the same directory:

```
python3 hello.py
```

The terminal prints:

```
Hello world
```

print()

`print()` writes values to the terminal.

```
print("Hello world")
print("The answer is", 42)
```

The output is:

```
Hello world
The answer is 42
```

Try with other strings and numbers.

input()

`input()` waits for the user and returns a string.

```
name = input("Name: ")
print("Hello,", name)
```

Example run:

```
Name: Ada
Hello, Ada
```

Use `int()` or `float()` when text input should become a number.

Variables

A variable name points to a value.

```
count = 3
price = 2.5
message = "samples"

print(count, message)
print(count * price)
```

You do not declare the type first. Python reads the value and keeps track of the type at runtime.

int Type

`int` means integer. You can use positive numbers, negative numbers, and zero.

```
print(123)
print(-321)
print(0)
```

You can also write binary, octal, and hexadecimal integers:

```
print(0b1010)
print(0o123)
print(0xABC)
```

float Type

`float` means a decimal number.

```
print(1.23)
print(-3.21)
print(0.0)
```

Scientific notation is useful for very large or very small numbers:

```
print(1.23e45)
print(6.78E-9)
```

Those mean 1.23×10^{45} and 6.78×10^{-9}.

str Type

`str` means string, or text.

```
course = "Python"
year = "2026"

print(course)
print(course + " " + year)
print(len(course))
```

Use quotes around string values. Both single quotes and double quotes are valid; choose one style and use it consistently.

bool Type and Comparisons

`bool` means `True` or `False`. Comparisons produce boolean values.

```
score = 82

print(score >= 60)
print(score == 100)
print(score != 0)
```

Common comparison operators are ==, !=, <, <=, >, and >=.

Arithmetic Operators

You can express arithmetic directly in Python.

```
print(3 + 4)
print(3 - 4)
print(3 * 4)
print(3 / 4)
```

These are equivalent to:

Example

$3 + 4$

$3 - 4$

3×4

$3 \div 4$

Power, Remainder, and Quotient

```
print(3 ** 4)   # power: 81
print(15 % 4)   # remainder: 3
print(15 / 4)   # normal division: 3.75
print(15 // 4)  # floor division: 3
```

Use // when you need an integer-like quotient. Use % when you need the remainder after division.

List Type

A list stores multiple values in one variable.

```
empty = []
numbers = [1, 2, 3]
mixed = [1, "two", 3.0]
nested = [1, [2, 3], 4]
```

Lists preserve order and can be changed after they are created.

```
numbers.append(4)
print(numbers)
print(len(numbers))
```

Indexing

Python counts from zero. A list with length n has indices from 0 to $n - 1$.

```
values = [5, 6, 7]

print(values[0])
print(values[1])
print(values[2])
```

Negative indices count from the end:

```
print(values[-1])
```

Slicing

A slice selects part of a sequence.

```
values = [10, 20, 30, 40, 50]

print(values[1:4])   # [20, 30, 40]
print(values[:3])    # [10, 20, 30]
print(values[3:])    # [40, 50]
print(values[::2])   # [10, 30, 50]
```

The stop index is not included. `values[1:4]` means index 1, index 2, and index 3.

Reading a File

Use `with open(...)` so Python closes the file automatically.

```
with open("names.txt", "r", encoding="utf-8") as handle:
    for line in handle:
        print(line.strip())
```

`line.strip()` removes the newline character at the end of each line.

Writing a File

Mode `"w"` creates a file or replaces an existing file.

```
with open("result.csv", "w", encoding="utf-8") as handle:
    handle.write("sample,count\n")
    handle.write("A,3\n")
    handle.write("B,5\n")
```

Use mode `"a"` when you want to append to the end of an existing file.

Working Directory

Relative paths are interpreted from the current working directory, not necessarily from the directory where the script is saved.

In class, keep scripts and practice files in one folder, then run:

Example

```
python3 script.py
```

from that folder. This makes file examples predictable.

if Statements

Use `if`, `elif`, and `else` to choose a path.

```
score = int(input("Score: "))

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("Keep practicing")
```

Indentation is part of Python syntax.

for Loops

A `for` loop repeats once for each item in a sequence.

```
names = ["Ada", "Grace", "Guido"]

for name in names:
    print("Hello,", name)
```

Use `range()` when you need repeated numbers:

```
for i in range(3):
    print(i)
```

while Loops

A `while` loop repeats while a condition is true.

```
count = 3

while count > 0:
    print(count)
    count = count - 1

print("Go")
```

Make sure something inside the loop changes the condition, otherwise the loop may never stop.

Defining Functions

A function packages reusable code behind a name.

```
def greet(name):
    print("Hello,", name)

greet("Ada")
greet("Grace")
```

Function names should describe the action they perform.

return Values

Use `return` when the function should give a value back.

```
def celsius_to_fahrenheit(celsius):
    return celsius * 9 / 5 + 32

temperature = celsius_to_fahrenheit(25)
print(temperature)
```

Printing and returning are different. Returning lets the caller keep using the result.

Exceptions

An exception means Python found a problem while running the program.

```
number = int("hello")
```

This raises `ValueError` because the string is not an integer.

When you see a traceback, start by reading the last line. It usually says the exception type and the immediate problem.

try and except

Use `try` and `except` when a failure is expected and you can recover from it.

```
text = input("Count: ")

try:
    count = int(text)
    print(count * 2)
except ValueError:
    print("Please type an integer.")
```

Catch specific exceptions so real bugs are still visible.

Importing Modules

A module is a file or package that provides Python code you can reuse.

```
import math

print(math.sqrt(9))
print(math.pi)
```

The Python standard library is installed with Python, so many useful modules require no extra installation.

Writing Your Own Module

Save helper functions in one file:

```
# tools.py
def square(x):
    return x * x
```

Import them from another file in the same folder:

```
# main.py
from tools import square

print(square(5))
```

Practice Exercise

Write `temperature.py`.

1. Ask the user for a Celsius temperature.
2. Convert the input to `float`.
3. Print the Fahrenheit temperature.
4. If the input is not a number, print a helpful message.

This exercise uses input, variables, arithmetic, functions, and exception handling together.